

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL AND DEBUGGING

OLEH:

ZAHRA NABILA

NIM 2310817320007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Zahra Nabila
NIM : 2310817320007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	7
B. Output Program	32
C. Pembahasan	35
D. Tautan Git	46
SOAL 2	47
A. Pembahasan	47

DAFTAR GAMBAR

Gambar 1. Contoh Penggunaan Debugger	6
Gambar 2. Screenshot Hasil Jawaban Soal 1	32
Gambar 3. Screenshot Hasil Jawaban Soal 1	32
Gambar 4. Screenshot Hasil Jawaban Soal 1	33
Gambar 5. Screenshot Hasil Debugging.....	34
Gambar 6. Screenshot Hasil Debugging Step Into	34
Gambar 7. Screenshot Hasil Debugging Step Over	35
Gambar 8. Screenshot Hasil Debugging Step Out	35

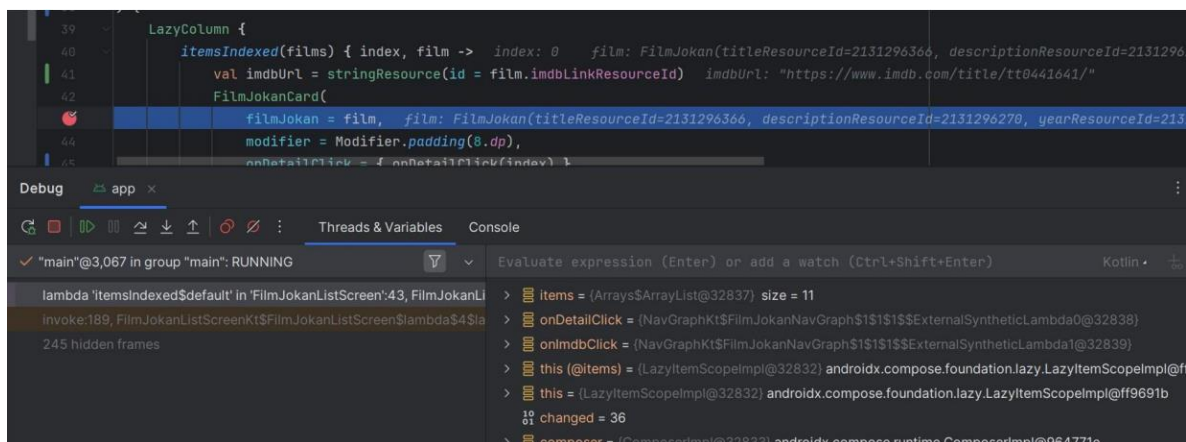
DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1	7
Table 2. Source Code Jawaban Soal 1	9
Table 3. Source Code Jawaban Soal 1	13
Table 4. Source Code Jawaban Soal 1	15
Table 5. Source Code Jawaban Soal 1	16
Table 6. Source Code Jawaban Soal 1	17
Table 7. Source Code Jawaban Soal 1	17
Table 8. Source Code Jawaban Soal 1	18
Table 9. Source Code Jawaban Soal 1	21
Table 10. Source Code Jawaban Soal 1	21
Table 11. Source Code Jawaban Soal 1	24
Table 12. Source Code Jawaban Soal 1	25
Table 13. Source Code Jawaban Soal 1	29
Table 14. Source Code Jawaban Soal 1	31

SOAL 1

1. Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:
 - a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
 - b. Gunakan ViewModelFactory dalam pembuatan ViewModel
 - c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment.
 - d. Gunakan logging untuk event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi. Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya. Berikut adalah contoh debugging dalam Android Studio.



Gambar 1. Contoh Penggunaan Debugger

A. Source Code

File MainActivity.kt	
1	package com.example.recycler_view
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	
6	class MainActivity : AppCompatActivity() {
7	override fun onCreate(savedInstanceState: Bundle?) {
8	super.onCreate(savedInstanceState)
9	setContentView(R.layout.activity_main)
10	
11	val fragmentManager = supportFragmentManager
12	val homeFragment = HomeFragment()
13	val fragment =
	fragmentManager.findFragmentByTag(HomeFragment::class.java
	.simpleName)
14	if (fragment !is HomeFragment) {
15	fragmentManager
16	.beginTransaction()
17	.add(R.id.frame_container, homeFragment,
	HomeFragment::class.java.simpleName)
18	.commit()
19	}
20	}
21	}

Table 1. Source Code Jawaban Soal 1

File DetailFragment.kt	
1	package com.example.recycler_view
2	

```

3  import android.os.Bundle
4  import android.util.Log
5  import android.view.LayoutInflater
6  import android.view.View
7  import android.view.ViewGroup
8  import androidx.fragment.app.Fragment
9  import androidx.fragment.app.viewModels
10 import androidx.lifecycle.lifecycleScope
11 import
    com.example.recycler_view.databinding.FragmentDetailBinding
12 import kotlinx.coroutines.flow.collectLatest
13 import kotlinx.coroutines.launch
14
15 class DetailFragment : Fragment() {
16
17     private var _binding: FragmentDetailBinding? = null
18     private val binding get() = _binding!!
19     private val viewModel: SeriesViewModel by viewModels {
20         SeriesViewModelFactory()
21     }
22
23     override fun onCreateView(
24         inflater: LayoutInflater, container: ViewGroup?,
25         savedInstanceState: Bundle?
26     ): View? {
27         _binding = FragmentDetailBinding.inflate(inflater,
28             container, false)
29         return binding.root
30     }
31
32     override fun onViewCreated(view: View,
33         savedInstanceState: Bundle?) {

```

30	super.onCreateView(view, savedInstanceState)
31	
32	lifecycleScope.launch {
33	viewModel.selectedSeries.collectLatest { series
->	
34	series?.let {
35	Log.d("DetailFragment", "Menampilkan detail untuk: \${series.title}")
36	binding.tvName.text = series.title
37	binding.tvPlot.text = series.plot
38	binding.tvYear.text = series.year
39	binding.tvCast.text = series.cast
40	binding.imgItemPhoto.setImageResource(series.photo)
41	}
42	}
43	}
44	}
45	
46	override fun onDestroyView() {
47	super.onDestroyView()
48	_binding = null
49	}
50	}

Table 2. Source Code Jawaban Soal 1

File HomeFragment.kt	
1	package com.example.recycler_view
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle

```

6  import android.util.Log
7  import android.view.LayoutInflater
8  import android.view.View
9  import android.view.ViewGroup
10 import androidx.fragment.app.Fragment
11 import androidx.fragment.app.viewModels
12 import androidx.lifecycle.lifecycleScope
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import
    com.example.recycler_view.databinding.FragmentHomeBinding
15 import kotlinx.coroutines.flow.collectLatest
16 import kotlinx.coroutines.launch
17
18 class HomeFragment : Fragment() {
19
20     private var _binding: FragmentHomeBinding? = null
21     private val binding get() = _binding!!
22     private val viewModel: SeriesViewModel by viewModels
    { SeriesViewModelFactory() }
23
24     private lateinit var seriesAdapter: ListSeriesAdapter
25
26     override fun onCreateView(
27         inflater: LayoutInflater, container: ViewGroup?,
28         savedInstanceState: Bundle?
29     ): View {
30         _binding = FragmentHomeBinding.inflate(inflater,
    container, false)
31         return binding.root
32     }
33

```

```

34         override fun onCreateView(view: View,
savedInstanceState: Bundle?) {
35             super.onCreateView(view, savedInstanceState)
36
37             seriesAdapter = ListSeriesAdapter(
38                 emptyList(),
39                 onWikiClick = { link ->
40                     Log.d("HomeFragment", "Tombol Intent
(WeTV) diklik: $link")
41                     val uri = Uri.parse(link)
42                     startActivity(Intent(Intent.ACTION_VIEW,
uri))
43                 },
44                 onDetailClick = { series ->
45                     Log.d("HomeFragment", "Tombol Detail
diklik: ${series.title}")
46                     viewModel.selectSeries(series)
47                     parentFragmentManager.beginTransaction()
48                         .replace(R.id.frame_container,
DetailFragment())
49                         .addToBackStack(null)
50                         .commit()
51                 }
52             )
53
54             binding.rvDrama.apply {
55                 layoutManager =
LinearLayoutManager(requireContext())
56                 setHasFixedSize(true)
57                 adapter = seriesAdapter
58             }

```

```

59
60         lifecycleScope.launch {
61             viewModel.seriesList.collectLatest {
updatedList ->
62                 seriesAdapter.updateList(updatedList)
63             }
64         }
65
66         if (viewModel.seriesList.value.isEmpty()) {
67             viewModel.setSeriesList(getSeriesList())
68         }
69     }
70
71     private fun getSeriesList(): List<Series> {
72         val dataTitle =
resources.getStringArray(R.array.data_name)
73         val dataLink =
resources.getStringArray(R.array.data_link)
74         val dataPhoto =
resources.obtainTypedArray(R.array.data_photo)
75         val dataPlot =
resources.getStringArray(R.array.data_plot)
76         val dataYear =
resources.getStringArray(R.array.data_year)
77         val dataCast =
resources.getStringArray(R.array.data_cast)
78
79         val list = ArrayList<Series>()
80         for (i in dataTitle.indices) {
81             val series = Series(
82                 dataTitle[i],

```

83	dataLink[i],
84	dataPhoto.getResourceId(i, -1),
85	dataPlot[i],
86	dataYear[i],
87	dataCast[i]
88	)
89	list.add(series)
90	}
91	dataPhoto.recycle()
92	return list
93	}
94	
95	override fun onDestroyView() {
96	super.onDestroyView()
97	_binding = null
98	}
99	}

Table 3. Source Code Jawaban Soal 1

File ListSeriesAdapter.kt	
1	package com.example.recycler_view
2	
3	import android.view.LayoutInflater
4	import android.view.View
5	import android.view.ViewGroup
6	import android.widget.Button
7	import android.widget.ImageView
8	import android.widget.TextView
9	import androidx.recyclerview.widget.RecyclerView
10	
11	class ListSeriesAdapter(

```

12     private var listSeries: List<Series>,
13     private val onWikiClick: (String) -> Unit,
14     private val onDetailClick: (Series) -> Unit
15 ) : RecyclerView.Adapter<ListSeriesAdapter.ViewHolder>()
16 {
17     class ViewHolder(itemView: View) :
18     RecyclerView.ViewHolder(itemView) {
19         val imgPhoto: ImageView =
20         itemView.findViewById(R.id.img_item_photo)
21         val tvTitle: TextView =
22         itemView.findViewById(R.id.tv_item_name)
23         val tvCast: TextView =
24         itemView.findViewById(R.id.tv_cast)
25         val btnWiki: Button =
26         itemView.findViewById(R.id.btn_wiki)
27         val btnDetail: Button =
28         itemView.findViewById(R.id.button_detail)
29     }
30
31     override fun onCreateViewHolder(parent: ViewGroup,
32     viewType: Int): ViewHolder {
33         val view =
34         LayoutInflater.from(parent.context).inflate(R.layout.item
35         _list, parent, false)
36         return ViewHolder(view)
37     }
38
39     override fun getItemCount(): Int = listSeries.size
40
41     override fun onBindViewHolder(holder: ViewHolder,

```

	position: Int) {
33	val series = listSeries[position]
34	holder.tvTitle.text = series.title
35	holder.tvCast.text = series.cast
36	holder.imgPhoto.setImageResource(series.photo)
37	holder.btnWiki.setOnClickListener {
	onWikiClick(series.link) }
38	holder.btnDetail.setOnClickListener {
	onDetailClick(series) }
39	}
40	
41	fun updateList(newList: List<Series>) {
42	listSeries = newList
43	notifyDataSetChanged()
44	}
45	}

Table 4. Source Code Jawaban Soal 1

File Series.kt	
1	package com.example.recycler_view
2	
3	import android.os.Parcelable
4	import kotlinx.parcelize.Parcelize
5	
6	@Parcelize
7	data class Series(
8	val title: String,
9	val link: String,
10	val photo: Int,
11	val plot: String,
12	val year: String,

13	val cast: String
14	) : Parcelable

Table 5. Source Code Jawaban Soal 1

File SeriesViewModel.kt	
1	package com.example.recycler_view
2	
3	import android.util.Log
4	import androidx.lifecycle.ViewModel
5	import kotlinx.coroutines.flow.MutableStateFlow
6	import kotlinx.coroutines.flow.StateFlow
7	
8	class SeriesViewModel : ViewModel() {
9	
10	private val _seriesList =
	MutableStateFlow<List<Series>>(emptyList())
11	val seriesList: StateFlow<List<Series>> get() =
	_seriesList
12	
13	private val _selectedSeries =
	MutableStateFlow<Series?>(null)
14	val selectedSeries: StateFlow<Series?> get() =
	_selectedSeries
15	
16	fun setSeriesList(list: List<Series>) {
17	_seriesList.value = list
18	Log.d("SeriesViewModel", "Data item masuk ke
	list: \$list")
19	}
20	
21	fun selectSeries(series: Series) {

22	<code>_selectedSeries.value = series</code>
23	<code>Log.d("SeriesViewModel", "Item dipilih:</code>
	<code>`\${series.title}`")</code>
24	<code>}</code>
25	<code>}</code>

Table 6. Source Code Jawaban Soal 1

File SeriesViewModelFactory.kt	
1	<code>package com.example.recycler_view</code>
2	
3	<code>import androidx.lifecycle.ViewModel</code>
4	<code>import androidx.lifecycle.ViewModelProvider</code>
5	
6	<code>class SeriesViewModelFactory : ViewModelProvider.Factory {</code>
7	<code> override fun <T : ViewModel> create(modelClass:</code>
	<code>Class<T>): T {</code>
8	<code> if</code>
	<code>(modelClass.isAssignableFrom(SeriesViewModel::class.java))</code>
9	<code>{</code>
10	<code> return SeriesViewModel() as T</code>
11	<code> }</code>
12	<code> throw IllegalArgumentException("Unknown ViewModel</code>
	<code>class")</code>
13	<code> }</code>
14	<code>}</code>

Table 7. Source Code Jawaban Soal 1

File activity_main.xml	
1	<code><androidx.constraintlayout.widget.ConstraintLayout</code>
	<code>xmlns:android="http://schemas.android.com/apk/res/androi</code>
	<code>d"</code>

2	xmlns:app="http://schemas.android.com/apk/res-auto"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	tools:context=".MainActivity">
7	
8	<FrameLayout
9	android:id="@+id/frame_container"
10	android:layout_width="match_parent"
11	android:layout_height="match_parent" />
12	</androidx.constraintlayout.widget.ConstraintLayout>

Table 8. Source Code Jawaban Soal 1

File fragment_detail.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
	"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fillViewport="true"
8	android:background="@drawable/bg"
9	tools:context=".DetailFragment">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:paddingBottom="32dp">
15	

16	<ImageView
17	android:id="@+id/img_item_photo"
18	android:layout_width="341dp"
19	android:layout_height="458dp"
20	android:layout_marginTop="32dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintEnd_toEndOf="parent"
23	app:layout_constraintStart_toStartOf="parent"
24	app:layout_constraintTop_toTopOf="parent"
25	tools:src="@drawable/antares" />
26	
27	<TextView
28	android:id="@+id/tv_name"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:layout_marginTop="16dp"
	android:fontFamily="serif"
32	android:textSize="25sp"
33	android:textStyle="bold"
34	app:layout_constraintEnd_toEndOf="parent"
35	app:layout_constraintStart_toStartOf="parent"
36	app:layout_constraintTop_toBottomOf="@id/img_
	item_photo"
37	tools:text="Judul Series" />
38	
39	<TextView
40	android:id="@+id/tv_year"
41	android:layout_width="wrap_content"
42	android:layout_height="wrap_content"
43	android:text="Tahun: "
44	android:fontFamily="serif"

45	android:textSize="18sp"
46	android:layout_marginTop="8dp"
47	app:layout_constraintTop_toBottomOf="@id/tv_n
48	ame"
49	app:layout_constraintStart_toStartOf="parent"
50	app:layout_constraintEnd_toEndOf="parent" />
51	<TextView
52	android:id="@+id/tv_cast"
53	android:layout_width="0dp"
54	android:layout_height="wrap_content"
55	android:text="Pemeran: "
56	android:fontFamily="serif"
57	android:textSize="18sp"
58	android:layout_marginTop="8dp"
59	android:layout_marginHorizontal="16dp"
60	android:textAlignment="center"
61	app:layout_constraintTop_toBottomOf="@id/tv_y
62	ear"
63	app:layout_constraintStart_toStartOf="parent"
64	app:layout_constraintEnd_toEndOf="parent" />
65	<TextView
66	android:id="@+id/tv_plot"
67	android:layout_width="0dp"
68	android:layout_height="wrap_content"
69	android:layout_margin="16dp"
70	android:fontFamily="serif"
71	android:text="Sinopsis: "
72	android:textAlignment="center"
73	android:textSize="18sp"

74	app:layout_constraintBottom_toBottomOf="paren
	t"
75	app:layout_constraintEnd_toEndOf="parent"
76	app:layout_constraintStart_toStartOf="parent"
77	
78	app:layout_constraintTop_toBottomOf="@+id/tv_cast" />
79	
80	</androidx.constraintlayout.widget.ConstraintLayout>
81	</ScrollView>

Table 9. Source Code Jawaban Soal 1

File fragment_home.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:background="@drawable/bg2">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/rvDrama"
11	android:layout_width="0dp"
12	android:layout_height="0dp"
13	app:layout_constraintTop_toTopOf="parent"
14	app:layout_constraintBottom_toBottomOf="parent"
15	app:layout_constraintStart_toStartOf="parent"
16	app:layout_constraintEnd_toEndOf="parent" />
17	</androidx.constraintlayout.widget.ConstraintLayout>

Table 10. Source Code Jawaban Soal 1

File item_list.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
	xmlns:android="http://schemas.android.com/apk/res/android
	"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	xmlns:tools="http://schemas.android.com/tools"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="8dp"
9	app:cardElevation="4dp">
10	
11	<androidx.constraintlayout.widget.ConstraintLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:padding="8dp"
15	tools:src="@drawable/bg2">
16	
17	<ImageView
18	android:id="@+id/img_item_photo"
19	android:layout_width="70dp"
20	android:layout_height="100dp"
21	android:scaleType="centerCrop"
22	app:layout_constraintStart_toStartOf="parent"
23	app:layout_constraintTop_toTopOf="parent"
	app:layout_constraintBottom_toBottomOf="parent"
24	"
	tools:src="@drawable/antares" />
25	
26	<TextView

27	android:id="@+id/tv_item_name"
28	android:layout_width="0dp"
29	android:layout_height="wrap_content"
30	android:layout_marginStart="12dp"
31	android:textSize="18sp"
32	android:textStyle="bold"
33	android:fontFamily="serif"
34	android:textColor="#000000"
35	app:layout_constraintStart_toEndOf="@id/img_it
36	em_photo"
	app:layout_constraintTop_toTopOf="parent"
37	app:layout_constraintEnd_toEndOf="parent"
38	tools:text="Antares" />
39	
40	<TextView
41	android:id="@+id/tv_item_plot"
42	android:layout_width="0dp"
43	android:layout_height="wrap_content"
44	android:layout_marginTop="4dp"
45	android:textSize="14sp"
46	android:fontFamily="serif"
47	app:layout_constraintTop_toBottomOf="@id/tv_it
48	em_name"
	app:layout_constraintStart_toStartOf="@id/tv_i
49	tem_name"
	app:layout_constraintEnd_toEndOf="parent"
50	tools:text="Cast: " />
51	
52	<Button
53	android:id="@+id/btn_wiki"
54	android:layout_width="wrap_content"

55	android:layout_height="wrap_content"
56	android:layout_marginStart="60dp"
57	android:layout_marginTop="12dp"
58	android:backgroundTint="#9bc6ec"
59	android:fontFamily="serif"
60	android:textStyle="bold"
61	android:text="WeTV"
62	app:layout_constraintStart_toStartOf="@id/tv_i
63	tem_plot"
64	em_plot" />
65	<Button
66	android:id="@+id/button_detail"
67	android:layout_width="wrap_content"
68	android:layout_height="wrap_content"
69	android:layout_marginStart="40dp"
70	android:backgroundTint="#9bc6ec"
71	android:fontFamily="serif"
72	android:textStyle="bold"
73	android:text="Detail"
74	app:layout_constraintStart_toEndOf="@id/btn_wi
75	ki"
76	app:layout_constraintTop_toTopOf="@id/btn_wiki
77	" />
78	</androidx.constraintlayout.widget.ConstraintLayout>
79	</androidx.cardview.widget.CardView>

Table 11. Source Code Jawaban Soal 1

File colors.xml	
1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	<color name="black">#FF000000</color>
4	<color name="white">#FFFFFFFF</color>
5	<color name="blue">#9bc6ec</color>
6	</resources>

Table 12. Source Code Jawaban Soal 1

File strings.xml	
1	<resources>
2	<string name="app_name">Rycycler View</string>
3	<string name="btn_detail">Detail</string>
4	<string name="btn_wiki">Wiki</string>
5	
6	<string-array name="data_name">
7	<item>Antares</item>
8	<item>Harus Kawin</item>
9	<item>Kaget Nikah</item>
10	<item>Jodoh atau Bukan</item>
11	<item>Kisah untuk Geri</item>
12	<item>Mozachiko</item>
13	<item>5 Detik dan Rasa Rindu</item>
14	<item>17 Selamanya</item>
15	<item>Teluk Alaska</item>
16	<item>2 Wajah Arjuna</item>
17	</string-array>
18	
19	<string-array name="data_photo">
20	<item>@drawable/antares</item>
21	<item>@drawable/harus_kawin</item>

22	<item>@drawable/kaget_nikah</item>
23	<item>@drawable/jodoh_bukan</item>
24	<item>@drawable/kisah_geri</item>
25	<item>@drawable/mozachiko</item>
26	<item>@drawable/rasa_rindu</item>
27	<item>@drawable/selamanya</item>
28	<item>@drawable/teluk_alaska</item>
29	<item>@drawable/wajah_arjuna</item>
30	</string-array>
31	
32	<string-array name="data_link">
33	<item>https://wetv.vip/id/play/a41kjezlclozg0i/y0040sv1hu 9-EP01A%3A_Antares</item>
34	<item>https://wetv.vip/id/play/hm0734w9etht4vn- Harus%20Kawin/w0047tjoaoi-EP01A%3A%20Harus%20Kawin</item>
35	<item>https://wetv.vip/id/play/yweofusr9c2jpy2/z0041b49t1 9-EP01A%3A_Kaget_Nikah</item>
36	<item>https://wetv.vip/id/play/k2cb8gl9t27lpbq- Jodoh%20atau%20Bukan/h0045cixsil- EP01A%3A%20Jodoh%20atau%20Bukan</item>
37	<item>https://wetv.vip/id/play/ns0pvt3754wv3il- Kisah%20Untuk%20Geri/p0036f3bfbj- EP01%3A%20Kisah%20Untuk%20Geri</item>
38	<item>https://wetv.vip/id/play/3qoivvzv2pbi6wt- Mozachiko/w0046wfrhc5-EP01A%3A%20Mozachiko</item>
39	<item>https://wetv.vip/id/play/yfxtx6zpngrg25j- 5%20Detik%20%26%20Rasa%20Rindu/b0047d2dh1h- EP01A%3A%205%20Detik%20%26%20Rasa%20Rindu</item>
40	<item>https://wetv.vip/id/play/2s6dyeoedxdi6xc- 17%20Selamanya/o00429wd0s1- EP01A%3A%2017%20Selamanya</item>

41	<code><item>https://wetv.vip/id/play/akwckof2lj9wjmn-Teluk%20Alaska/i0041ztcg2t-EP01A%3A%20Teluk%20Alaska</item></code>
42	<code><item>https://wetv.vip/id/play/ff3culwgrhb8q63-Dua%20Wajah%20Arjuna/k0047b3g69u-EP01A%3A%20Dua%20Wajah%20Arjuna</item></code>
43	<code></string-array></code>
44	
45	<code><string-array name="data_plot"></code>
46	<code><item>Series ini mengisahkan Zea, siswi baru yang berusaha mengungkap kebenaran di balik kecelakaan kakaknya yang melibatkan geng motor bernama Calderioz, yang dipimpin oleh Antares.</item></code>
47	<code><item>Mengisahkan lima sahabat yang dipaksa menikah oleh keluarga mereka. Serial ini menyoroti dilema dan tekanan sosial yang mereka hadapi dalam menghadapi pernikahan yang tidak diinginkan.</item></code>
48	<code><item>Lalita mengalami kecelakaan yang menyebabkan kehilangan keperawanannya dan terpaksa menikah dengan Andre, pemuda yang menolongnya. Pernikahan ini dijalani tanpa cinta karena desakan orang tua.</item></code>
49	<code><item>Natalie, seorang wanita stylish, hidupnya berubah saat ibunya menjodohkannya dengan anak partner bisnisnya. Serial ini mengeksplorasi dilema antara cinta dan perjodohan.</item></code>
50	<code><item>Setelah keluarganya bangkrut, Dinda, yang dulunya populer, harus menghadapi realitas baru dan menjalin hubungan dengan Geri, mantan musuhnya, dalam sebuah kontrak palsu yang berubah menjadi cinta.</item></code>
51	<code><item>Moza, siswi culun, bertekad membuat Chiko, siswa populer, jatuh cinta padanya dalam 100 hari.</item></code>

52	<code><item>Via, seorang karyawan muda, mencari artis baru untuk duet dengan Drupadi. Gana pun ditunjuk sebagai teman duet, dan mereka merilis singel yang diadaptasi dari puisi karya Via.</item></code>
53	<code><item>Dawai, gadis polos yang disukai Putra, menyimpan rahasia besar terkait masa lalu mereka yang memengaruhi hubungan mereka.</item></code>
54	<code><item>Anastasia, gadis pendiam, bertemu kembali dengan Alister, teman masa kecilnya yang kini menjadi siswa populer.</item></code>
55	<code><item>Arjuna, pria dengan wajah biasa, menemukan spray misterius yang mengubah wajahnya menjadi tampan, mengubah kehidupannya secara drastis.</item></code>
56	<code></string-array></code>
57	
58	<code><string-array name="data_year"></code>
59	<code><item>30 Juli 2021</item></code>
60	<code><item>5 Januari 2024</item></code>
61	<code><item>23 Desember 2021</item></code>
62	<code><item>13 Januari 2023</item></code>
63	<code><item>5 Maret 2021</item></code>
64	<code><item>2 Juni 2023</item></code>
65	<code><item>8 September 2023</item></code>
66	<code><item>10 Maret 2022</item></code>
67	<code><item>5 November 2021</item></code>
68	<code><item>30 Oktober 2023</item></code>
69	<code></string-array></code>
70	
71	<code><string-array name="data_cast"></code>
72	<code><item>Angga Yunanda, Beby Tsabina, Irzan Faiq</item></code>

73	<code><item>Kevin Julio, Haico Van der Veken</item></code>
74	<code><item>Aurora Ribero, Kevin Julio, Steffi Zamora, Fero Walandouw</item></code>
75	<code><item>Megan Domani, Rayn Wijaya, Salshabilla Adriani</item></code>
76	<code><item>Angga Yunanda, Syifa Hadju </item></code>
77	<code><item>Rebecca Kloppe, Junior Roberts, Yesaya Abraham</item></code>
78	<code><item>Prilly Latuconsina, Bryan Domani, Salshabilla Adriani</item></code>
79	<code><item>Syifa Hadju, Rizky Nazar, Steffi Zamora</item></code>
80	<code><item>Syifa Hadju, Devano Danendra, Kaneishia Yusuf</item></code>
81	<code><item>Yesaya Abraham, Beby Tsabina, Mikha Herman</item></code>
82	<code></string-array></code>
83	
84	<code><string name="hello_blank_fragment">Hello blank fragment</string></code>
85	<code></resources></code>

Table 13. Source Code Jawaban Soal 1

File build.gradle.kts (:app)	
1	<code>plugins {</code>
2	<code>id("com.android.application")</code>
3	<code>id("org.jetbrains.kotlin.android")</code>
4	<code>id("org.jetbrains.kotlin.plugin.parcelize")</code>
5	<code>}</code>
6	
7	<code>android {</code>

```

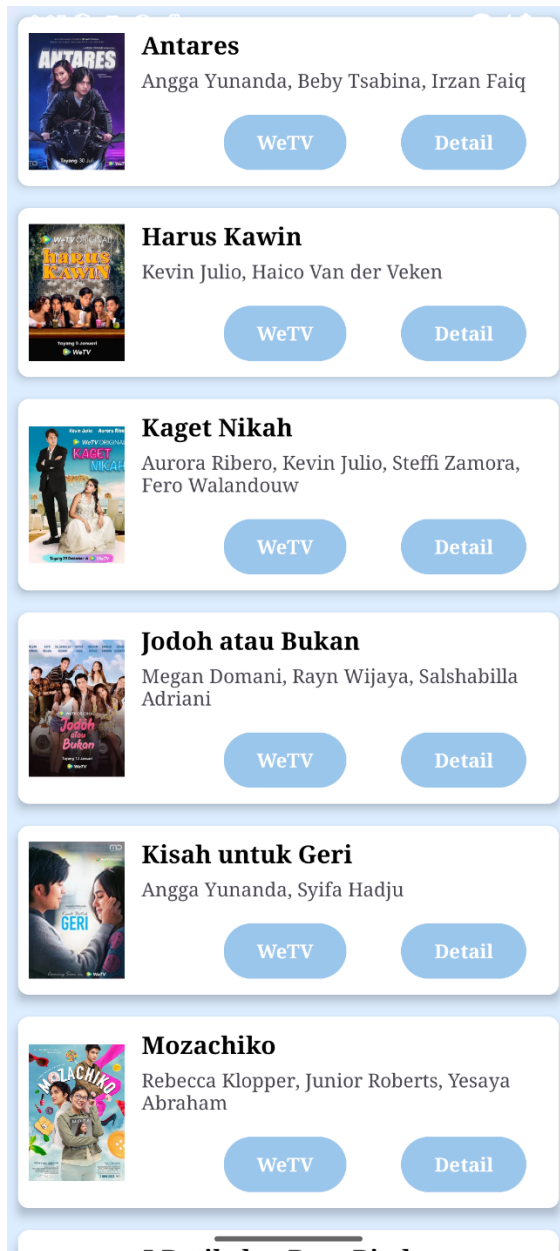
8      namespace = "com.example.recycler_view"
9      compileSdk = 35
10
11     defaultConfig {
12         applicationId = "com.example.recycler_view"
13         minSdk = 30
14         targetSdk = 35
15         versionCode = 1
16         versionName = "1.0"
17
18         testInstrumentationRunner =
19         "androidx.test.runner.AndroidJUnitRunner"
20     }
21     buildTypes {
22         release {
23             isMinifyEnabled = false
24             proguardFiles(
25                 getDefaultProguardFile("proguard-android-
26                 optimize.txt"),
27                 "proguard-rules.pro"
28             )
29         }
30     compileOptions {
31         sourceCompatibility = JavaVersion.VERSION_11
32         targetCompatibility = JavaVersion.VERSION_11
33     }
34     kotlinOptions {
35         jvmTarget = "11"
36     }

```

37	buildFeatures {
38	viewBinding = true
39	}
40	}
41	
42	dependencies {
43	
44	implementation(libs.androidx.core.ktx)
45	implementation(libs.androidx.appcompat)
46	implementation(libs.material)
47	implementation(libs.androidx.activity)
48	implementation("androidx.recyclerview:recyclerview:1.3.1"
	)
49	implementation("androidx.constraintlayout:constraintlayout:2.1.4")
50	
51	implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1")
52	implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.6.1")
53	implementation("androidx.fragment:fragment-ktx:1.5.7")
54	
55	testImplementation(libs.junit)
56	androidTestImplementation(libs.androidx.junit)
57	androidTestImplementation(libs.androidx.espresso.core)
58	}

Table 14. Source Code Jawaban Soal 1

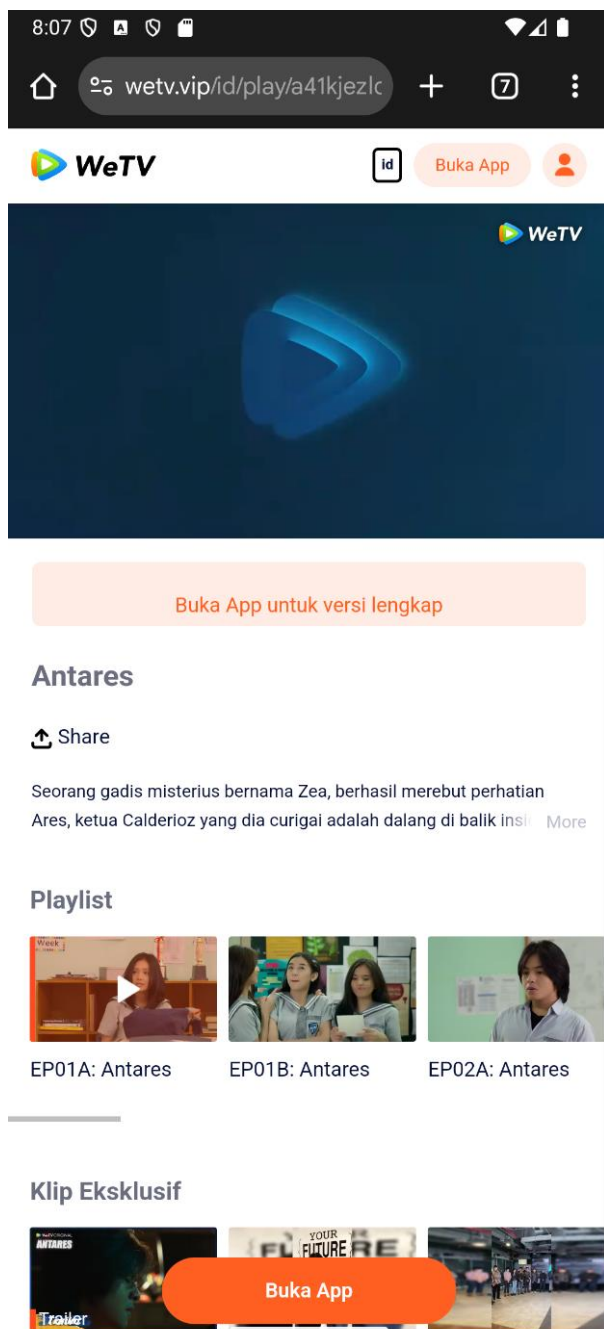
B. Output Program



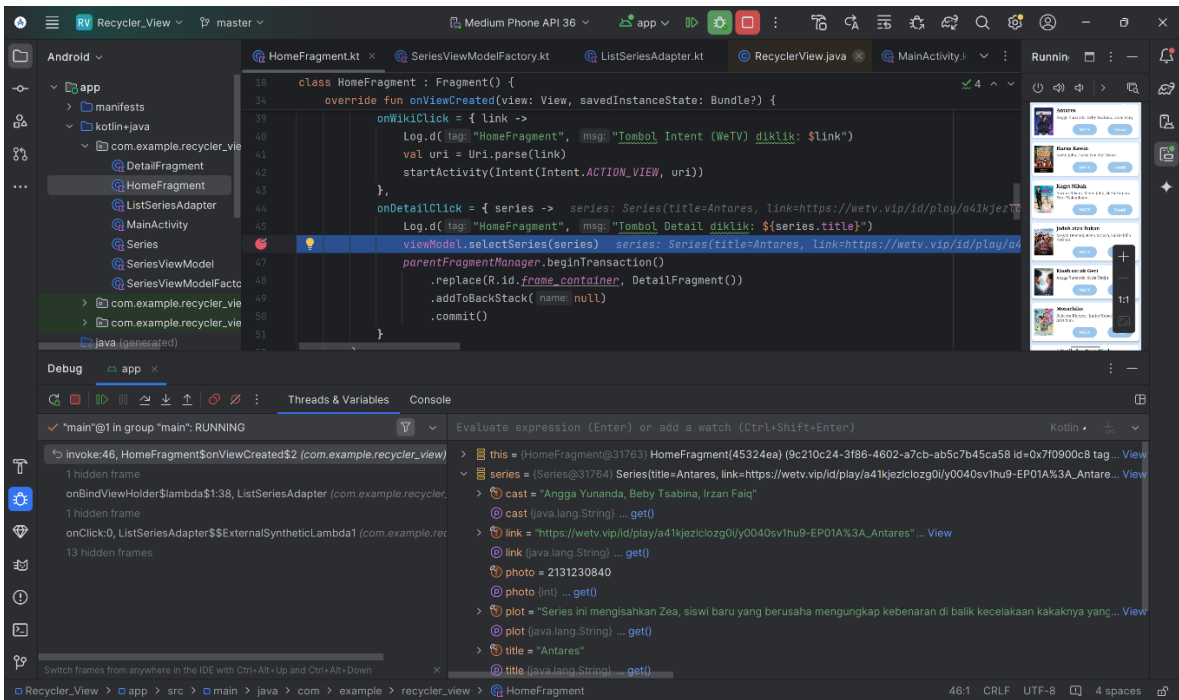
Gambar 2. Screenshot Hasil Jawaban Soal 1



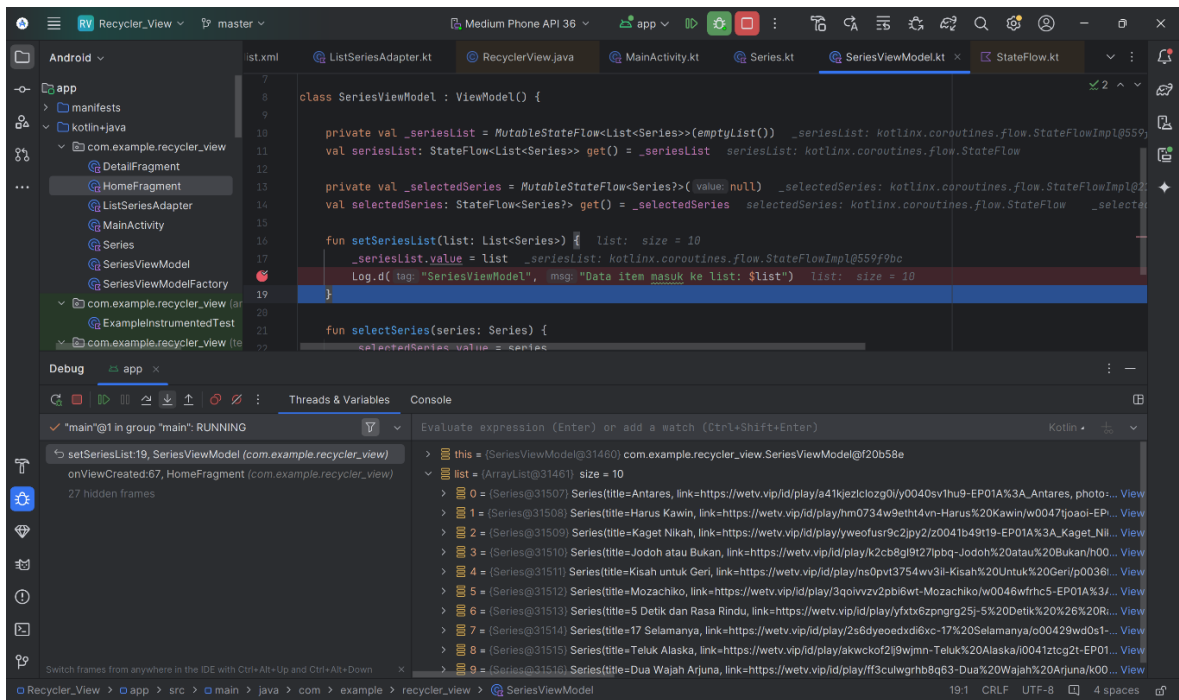
Gambar 3. Screenshot Hasil Jawaban Soal 1



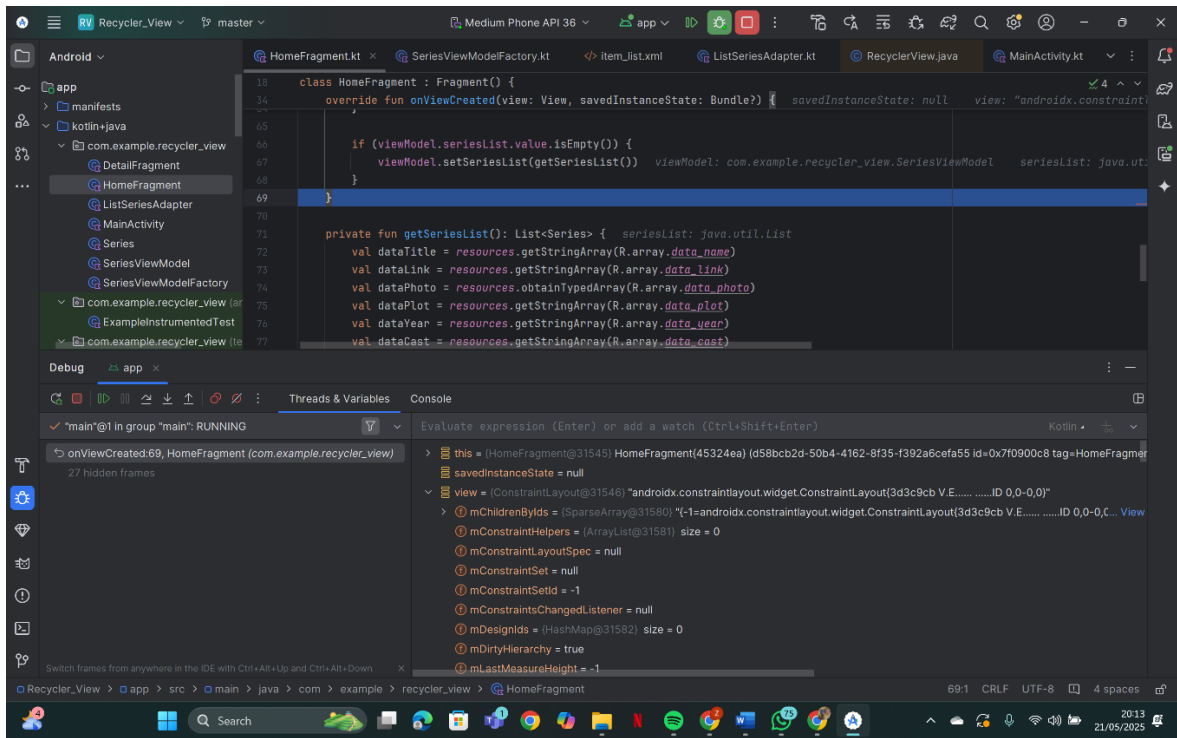
Gambar 4. Screenshot Hasil Jawaban Soal 1



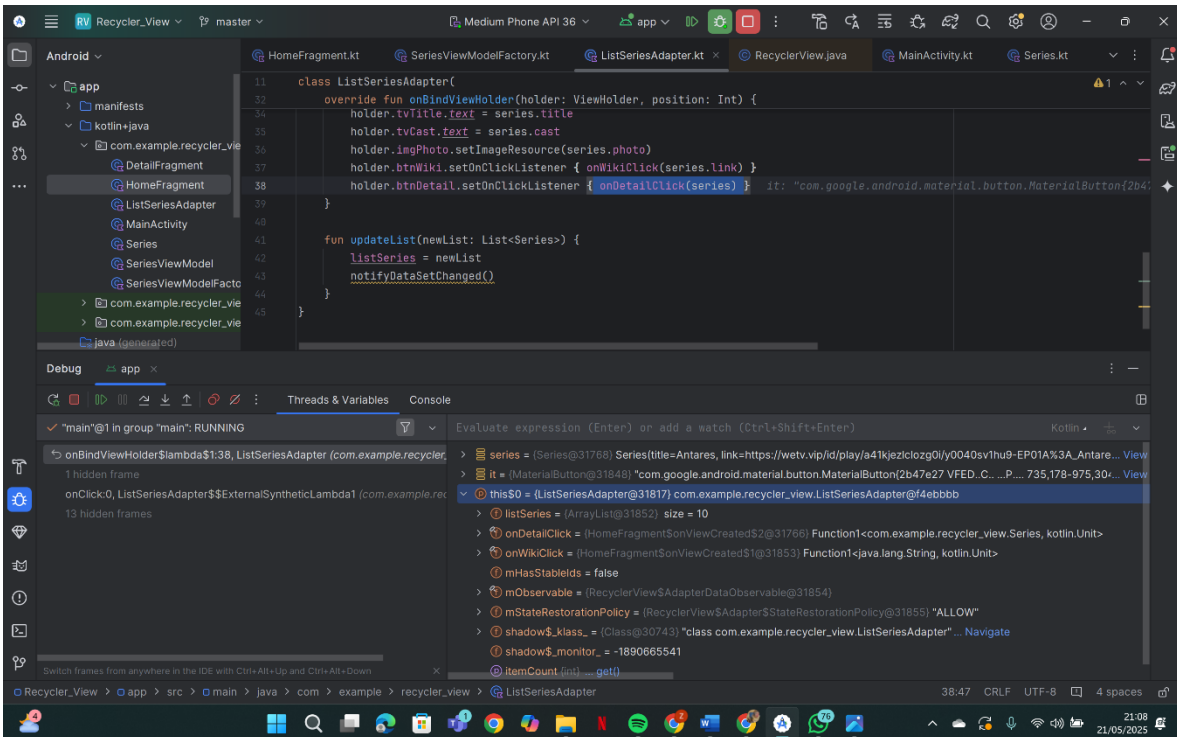
Gambar 5. Screenshot Hasil Debugging



Gambar 6. Screenshot Hasil Debugging Step Into



Gambar 7. Screenshot Hasil Debugging Step Over



Gambar 8. Screenshot Hasil Debugging Step Out

C. Pembahasan

File MainActivity.kt

Kelas **MainActivity** mewarisi dari **AppCompatActivity**, yang merupakan superclass umum untuk aktivitas yang menggunakan fitur-fitur modern seperti **ActionBar**. Di dalam metode **onCreate()**, pertama-tama dipanggil **super.onCreate(savedInstanceState)** untuk memastikan bahwa logika inisialisasi dari superclass berjalan sebagaimana mestinya. Selanjutnya, **setContentView(R.layout.activity_main)** digunakan untuk menentukan layout XML yang akan digunakan sebagai tampilan utama dari aktivitas ini, dalam hal ini adalah **activity_main.xml**.

Setelah layout diatur, kode melanjutkan dengan inisialisasi **FragmentManager** melalui **supportFragmentManager**. **FragmentManager** ini digunakan untuk mengelola transaksi fragment seperti penambahan, penggantian, atau penghapusan fragment. Kemudian, dibuat sebuah *instance* dari **HomeFragment**, yaitu fragment yang akan ditampilkan dalam activity. Baris **val fragment = fragmentManager.findFragmentByTag(HomeFragment::class.java.simpleName)** digunakan untuk memeriksa apakah fragment dengan tag tertentu yang berdasarkan nama kelas **HomeFragment** sudah ada atau belum dalam manajer fragment.

Jika fragment yang ditemukan bukan *instance* dari **HomeFragment** (**fragment !is HomeFragment**), maka dilakukan transaksi untuk menambahkan fragment ke dalam container dengan ID **R.id.frame_container**. Metode **beginTransaction()** digunakan untuk memulai transaksi, kemudian **add()** menambahkan fragment ke dalam layout. Parameter **HomeFragment::class.java.simpleName** sebagai tag digunakan agar fragment ini bisa dikenali atau dicari kembali di kemudian hari. Terakhir, transaksi ini dikonfirmasi dan dijalankan dengan **commit()**.

File DetailFragment.kt

DetailFragment bertanggung jawab untuk menampilkan detail dari suatu series. **DetailFragment** mewarisi dari kelas **Fragment**, dan menggunakan *view binding*

untuk mengakses elemen-elemen UI secara aman dan efisien tanpa perlu menggunakan **findViewById**.

View Binding dilakukan melalui objek **_binding** yang bertipe *nullable* dan diinisialisasi di dalam metode **onCreateView()** dengan memanggil **FragmentDetailBinding.inflate(...)**. Objek ini kemudian diakses menggunakan properti **binding** yang dijamin non-null selama siklus hidup view masih aktif.

Selain View Binding, **DetailFragment** juga terintegrasi dengan **SeriesViewModel**, yaitu sebuah ViewModel yang mengelola data series secara terpisah dari UI dan menyediakannya melalui **StateFlow** bernama **selectedSeries**. ViewModel ini dibuat menggunakan **SeriesViewModelFactory** untuk memastikan dependensi yang dibutuhkan tersedia. Ketika view fragment telah dibuat (**onViewCreated()**), fragment mulai mengobservasi **selectedSeries** menggunakan coroutine dalam **lifecycleScope.launch**. Penggunaan **collectLatest** memastikan bahwa setiap kali ada data series baru yang dipilih, UI akan diperbarui secara otomatis. Jika data yang diterima tidak null, maka informasi seperti **title**, **plot**, **year**, **cast**, dan **photo** dari series tersebut akan ditampilkan pada elemen-elemen UI yang sesuai menggunakan binding. Untuk mencegah terjadinya memory leak, fragment menerapkan praktik yang baik dengan menyetel **_binding** ke null di dalam metode **onDestroyView()**.

File HomeFragment.kt

Fragment ini memanfaatkan fitur View Binding melalui **FragmentHomeBinding**, yang memberikan akses langsung dan aman ke elemen-elemen antarmuka pengguna tanpa harus menggunakan **findViewById()**. Dalam metode **onCreateView()**, layout fragment di-*inflate* dan dikembalikan sebagai root view. Kemudian, pada **onViewCreated()**, **RecyclerView** diatur menggunakan **LinearLayoutManager** untuk menampilkan daftar secara vertikal, dan dioptimalkan dengan **setHasFixedSize(true)** agar performanya lebih baik jika ukuran item tetap.

Sebuah adapter bernama **ListSeriesAdapter** dibuat dan dikaitkan dengan **RecyclerView**. Adapter ini menerima dua aksi **callback** melalui **onWikiClick**, yang akan membuka tautan WeTV dari sebuah series menggunakan **Intent** dan browser eksternal, serta **onDetailClick**, yang akan membuat instance **DetailFragment**. Pada saat detail diklik, data series yang dipilih dikirim ke **SeriesViewModel** melalui fungsi **selectSeries()**. Kemudian, **DetailFragment** ditampilkan menggunakan **FragmentManager**, dan ditambahkan ke back stack agar pengguna dapat kembali ke daftar sebelumnya.

Untuk mengisi data, **HomeFragment** mengambil informasi series dari resource XML melalui metode **getSeriesList()**. Data tersebut mencakup judul, tautan, gambar, alur cerita, tahun rilis, dan pemeran, yang dikemas ke dalam objek **Series**. Resource berupa **TypedArray** yang digunakan untuk gambar juga direcycle setelah selesai digunakan untuk menghindari kebocoran memori. Setelah data siap, jika **seriesList** pada **ViewModel** masih kosong, maka data dari resource akan dimasukkan ke dalam **ViewModel** menggunakan **setSeriesList()**.

Selain itu, fragment ini juga mengobservasi data dari **seriesList** di **ViewModel** menggunakan coroutine dalam **lifecycleScope.launch**. Setiap kali data diperbarui, adapter akan memanggil **updateList()** untuk menampilkan daftar yang baru. Terakhir, dalam **onDestroyView()**, binding disetel ke **null** untuk memastikan referensi ke tampilan dilepas ketika fragment dihancurkan, sehingga mencegah terjadinya memory leak.

File ListSeriesAdapter.kt

Kelas ini menampilkan daftar objek **Series** dalam bentuk daftar item. Adapter ini menerima tiga parameter utama dalam konstruktor sebuah **List<Series>** yang berisi data drama, sebuah lambda **onWikiClick** yang menangani aksi saat pengguna ingin membuka tautan WeTV, dan **onDetailClick** yang menangani aksi ketika pengguna ingin melihat detail drama.

Di dalam adapter terdapat kelas **ViewHolder**, yang bertanggung jawab untuk menampung dan menginisialisasi elemen-elemen UI dari setiap item, yaitu **ImageView**

untuk **imgPhoto**, dua **TextView** untuk **tvTitle**, dan **tvPlot**, serta dua **Button** masing-masing untuk membuka **btnWiki** dan **btnDetail**. Layout setiap item dibuat dengan menggunakan file **item_list.xml** yang di-*inflate* di dalam metode **onCreateViewHolder()**. Metode ini mengembalikan objek **ViewHolder** yang siap digunakan.

Metode **getItemCount()** akan mengembalikan jumlah total item dalam daftar **listSeries**, yang menentukan seberapa banyak item yang akan ditampilkan di layar. Kemudian, pada **onBindViewHolder()**, adapter akan mengikat data ke elemen UI berdasarkan posisi item dalam daftar. Di sini, elemen-elemen seperti **title**, **photo**, dan **cast** dimasukkan ke tampilan masing-masing, dan tombol-tombol diatur dengan listener yang memanggil fungsi callback yang telah didefinisikan. Misalnya, **btnWiki** akan membuka link WeTV menggunakan fungsi **onWikiClick**, sedangkan **btnDetail** akan memicu **onDetailClick** yang akan membawa data drama ke tampilan detail.

Untuk mendukung pembaruan data, adapter menyediakan fungsi **updateList()**, yang akan mengganti isi **listSeries** dengan daftar baru, lalu memanggil **notifyDataSetChanged()** untuk memberi tahu **RecyclerView** bahwa data telah berubah dan tampilan harus diperbarui. Dengan desain seperti ini, **ListSeriesAdapter** mampu menampilkan dan mengelola daftar series secara dinamis dan interaktif dalam aplikasi.

File Series.kt

Kelas ini menggunakan anotasi **@Parcelize** dan mengimplementasikan antarmuka **Parcelable**, yang memungkinkan objek **Series** untuk dikirim antar komponen Android (seperti antar **Activity** atau **Fragment**) melalui **Bundle**.

Setiap instance dari **Series** memiliki enam property yaitu, **title** bertipe **String** untuk menyimpan judul serial, **link** untuk menyimpan URL untuk mengarah ke web WeTV, **photo** bertipe **Int** yang merupakan resource ID dari drawable, **plot** yang berisi sinopsis cerita, **year** untuk tahun rilis, dan **cast** yang memuat daftar pemeran.

Dengan penggunaan **@Parcelize**, proses pembuatan parcel (objek yang bisa dipaketkan) menjadi otomatis, sehingga tidak perlu menuliskan kode boilerplate seperti **writeToParcel()** atau **describeContents()**.

File **SeriesViewModel.kt**

Kelas **SeriesViewModel** adalah kelas turunan dari **ViewModel** yang bertanggung jawab untuk menyimpan dan mengelola data terkait daftar dan item **Series** yang dipilih dalam aplikasi. **ViewModel** ini memanfaatkan **StateFlow** dari Kotlin Coroutines untuk menyediakan data secara reaktif dan dapat diobservasi oleh komponen UI seperti **Fragment**. Properti **_seriesList** bertipe **MutableStateFlow<List<Series>>** menyimpan seluruh daftar series, sedangkan **seriesList** yang diekspos ke luar sebagai **StateFlow** digunakan agar data dapat diamati tanpa bisa diubah dari luar. Dengan pendekatan ini, data daftar series akan tetap konsisten meskipun konfigurasi ulang seperti rotasi layar terjadi.

Selain itu, **ViewModel** juga menyimpan state **Series** yang sedang dipilih dalam properti **_selectedSeries**, dan menyediakan versi publik **selectedSeries** yang bersifat read-only. Fungsi **setSeriesList()** digunakan untuk menetapkan atau memperbarui isi dari daftar series yang dimiliki **ViewModel**. Fungsi ini juga menuliskan log untuk membantu proses debugging. Sementara itu, fungsi **selectSeries()** digunakan untuk menetapkan satu item **Series** yang sedang dipilih, dan juga mencatat informasi item yang dipilih ke dalam log. Pendekatan ini memungkinkan fragment seperti **HomeFragment** dan **DetailFragment** untuk saling berbagi data melalui **ViewModel** dengan cara yang aman, efisien, dan sesuai arsitektur Android modern.

File **SeriesViewModelFactory.kt**

Kelas **SeriesViewModelFactory** merupakan implementasi dari antarmuka **ViewModelProvider.Factory** yang digunakan untuk membuat instance dari **SeriesViewModel**. **Factory** ini diperlukan karena **ViewModel** tidak boleh langsung diinstansiasi menggunakan constructor biasa, terutama saat menggunakan mekanisme **by**

viewModels di fragment atau activity, agar pembuatan **ViewModel** tetap terpisah dari komponen UI dan mengikuti prinsip dependency injection.

Metode **create()** di dalam factory ini akan dipanggil oleh sistem saat sebuah **ViewModel** dibutuhkan. Fungsi ini menerima parameter **modelClass**, yaitu kelas **ViewModel** yang diminta. Di dalamnya dilakukan pengecekan apakah **modelClass** sesuai dengan **SeriesViewModel**. Jika cocok, maka instance baru dari **SeriesViewModel** dibuat dan dikembalikan dengan casting ke tipe generik **T**. Jika tidak cocok, maka factory akan melemparkan **IllegalArgumentException** sebagai penanda bahwa kelas yang diminta tidak dikenali. Dengan menggunakan **SeriesViewModelFactory**, pembuatan **ViewModel** menjadi lebih fleksibel dan aman, serta mendukung penerapan arsitektur MVVM secara bersih.

File activity_main.xml

Kode XML tersebut merupakan layout utama dari **MainActivity** dalam aplikasi Android yang menggunakan **ConstraintLayout** sebagai root layout. Di dalamnya terdapat satu komponen penting, yaitu **FrameLayout** dengan ID **@+id/frame_container**. Elemen ini berfungsi sebagai **wadah** untuk menampung dan menampilkan **Fragment**, misalnya **HomeFragment** atau **DetailFragment**, yang dimuat secara dinamis melalui transaksi fragment dari kode Kotlin.

Tag **ConstraintLayout** menyediakan kemampuan untuk menempatkan dan mengatur posisi elemen-elemen UI secara fleksibel menggunakan constraint antar komponen atau ke parent layout. Namun dalam kasus ini, hanya ada satu elemen di dalamnya, yaitu **FrameLayout**. Meskipun begitu, karena ukuran **FrameLayout** diatur ke **match_parent** baik untuk lebar maupun tinggi, maka elemen ini akan mengisi seluruh ruang yang tersedia dalam layar.

Atribut **tools:context=".MainActivity"** digunakan oleh Android Studio saat preview layout di editor, untuk memberi tahu bahwa layout ini digunakan dalam **MainActivity**.

File fragment_detail.xml

Layout ini menggunakan **ScrollView** sebagai root. Di dalam **ScrollView**, terdapat **ConstraintLayout** sebagai container utama untuk menata elemen-elemen UI secara fleksibel. Elemen pertama adalah **ImageView** dengan ID **img_item_photo**, yang menampilkan gambar serial dengan ukuran tetap (**341dp x 458dp**) dan **scaleType centerCrop** agar gambar terisi penuh namun tetap proporsional. Gambar ini ditampilkan di bagian atas layout dan diberi margin atas **32dp**. Terdapat empat buah **TextView** yaitu, **tv_name** untuk menampilkan nama atau judul serial, ukuran teks **25sp, bold**, dan menggunakan font **serif** agar tampak khas seperti judul buku atau film, **tv_year** untuk menampilkan tahun rilis dengan label “Tahun:”, **tv_cast** untuk menampilkan pemeran dari serial, diletakkan tepat di bawah tahun dan dipusatkan, **tv_plot** untuk menampilkan sinopsis atau alur cerita dari serial tersebut, juga dipusatkan dan diberi margin agar tampak rapi.

Semua elemen diposisikan menggunakan **ConstraintLayout**, yaitu dengan menentukan hubungan seperti **app:layout_constraintTop_toBottomOf**, **layout_constraintStart_toStartOf**, dan **layout_constraintEnd_toEndOf** agar konten tersusun secara vertikal dan simetris di tengah layar.

Atribut **android:background="@drawable/bg"** pada **ScrollView** menunjukkan bahwa latar belakang layar menggunakan gambar atau bentuk dari resource drawable dengan nama **bg**.

File fragment_home.xml

Kelas ini root layout-nya menggunakan **ConstraintLayout**, yang memberi fleksibilitas tinggi dalam menyusun elemen-elemen UI. Layout ini juga memiliki latar belakang bergambar melalui atribut **android:background="@drawable/bg2"**, yang memberikan estetika visual yang lebih menarik dibandingkan latar belakang polos.

Satu-satunya elemen yang dimuat dalam **ConstraintLayout** ini adalah **RecyclerView** dengan ID **rvDrama**. Komponen ini digunakan untuk menampilkan daftar series secara efisien, karena hanya memuat tampilan yang terlihat di layar dan

mendaur ulang elemen lainnya. **RecyclerView** diatur agar mengisi seluruh area layar dengan menggunakan constraint ke setiap **Top**, **Bottom**, **Start**, **End**, serta lebar dan tinggi diset ke **0dp** yang berarti akan mengikuti constraint yang telah didefinisikan.

Layout ini bekerja bersama dengan **DramaAdapter** dan data dari **Series** untuk menampilkan daftar item yang interaktif, di mana pengguna bisa menekan tombol untuk membuka halaman detail atau tautan WeTV dari serial tersebut.

File item_list.xml

Root elemen dari layout ini adalah **CardView**, yang memberikan tampilan kartu dengan sudut melengkung dengan menggunakan **cardCornerRadius="8dp"** dan bayangan dengan menggunakan **cardElevation="4dp"**, menciptakan tampilan modern dan terangkat dari latar belakang.

Di dalam **CardView**, terdapat **ConstraintLayout** sebagai wadah utama untuk mengatur tata letak elemen UI seperti gambar, teks, dan tombol dengan posisi yang fleksibel dan presisi. Bagian kiri layout menampilkan **ImageView** (**@+id/img_item_photo**) berukuran **70x100dp** yang menampilkan gambar sampul serial dengan skala **centerCrop**, sehingga gambar akan memenuhi area tampilan secara proporsional.

Di samping gambar, terdapat dua **TextView**: satu untuk nama series dengan menggunakan **@+id/tv_item_name** dengan ukuran teks besar dan gaya tebal untuk menonjolkan judul, dan satu lagi di bawahnya yaitu **@+id/tv_item_plot** yang menampilkan informasi singkat seperti nama pemeran. Kedua teks ini menggunakan font **serif**, memberi nuansa klasik dan rapi. Di bagian bawah, terdapat dua tombol **Button** yang berfungsi sebagai aksi untuk pengguna. **btn_wiki** diberi label "WeTV" dan kemungkinan digunakan untuk membuka link eksternal yaitu WeTV. Tombol kedua yaitu **button_detail** membuka halaman detail serial. Kedua tombol ini didesain dengan **backgroundTint** berwarna biru muda dengan kode warna **#9bc6ec**, sehingga tampak serasi.

File colors.xml

Dalam kelas ini terdapat beberapa warna yang didefinisikan untuk digunakan secara konsisten di seluruh aplikasi, yaitu:

- **black** – Diberi nilai **#FF000000**, yang merupakan warna hitam sepenuhnya.
- **white** – Diberi nilai **#FFFFFFFF**, warna putih dengan opasitas penuh.
- **blue** – Diberi nilai **#9bc6ec**, warna biru muda yang sering dipakai dalam elemen UI seperti latar belakang tombol (**backgroundTint**) agar terlihat segar dan menarik secara visual.

File strings.xml

Terdapat tiga string tunggal yaitu **app_name**, **btn_detail**, dan **btn_wiki** yang masing-masing digunakan sebagai nama aplikasi, label tombol detail, dan tombol WeTV. Selanjutnya, file ini berisi beberapa **string-array** yang digunakan untuk menampilkan data dinamis dalam daftar seperti **RecyclerView**. **data_name** berisi daftar judul drama, **data_plot** berisi sinopsis singkat tiap drama, **data_year** mencatat tanggal rilisnya, **data_cast** menampilkan nama-nama pemeran utama, dan **data_photo** untuk menampilkan poster tiap series. Masing-masing elemen dalam array ini disusun secara paralel, artinya urutannya harus konsisten satu sama lain agar data bisa ditampilkan dengan benar.

File build.gradle.kts (:app)

Pada file **build.gradle.kts** (:app) yang ditampilkan merupakan berkas konfigurasi utama untuk aplikasi Android berbasis Kotlin yang menggunakan DSL (*Domain Specific Language*) Kotlin. File ini berperan dalam mendefinisikan proses build aplikasi, termasuk plugin yang digunakan, konfigurasi SDK Android, fitur build, serta daftar dependensi. Di bagian atas file, terdapat deklarasi tiga plugin utama: **com.android.application**, **org.jetbrains.kotlin.android**, dan **org.jetbrains.kotlin.plugin.parcelize**. Plugin **com.android.application** digunakan untuk menyatakan bahwa modul ini adalah aplikasi Android. Plugin **kotlin.android** memungkinkan penggunaan bahasa Kotlin

dalam pengembangan Android, sementara plugin **kotlin-parcelize** memungkinkan penggunaan anotasi **@Parcelize** untuk mempermudah implementasi **Parcelable**.

Blok **android** memuat konfigurasi inti proyek. Properti **namespace** diatur ke **"com.example.recycler_view"**, yang berfungsi sebagai ruang nama unik aplikasi. Versi **compileSdk**, **targetSdk**, dan **minSdk** masing-masing ditetapkan ke 35, 35, dan 30. Ini menandakan bahwa aplikasi dikembangkan menggunakan API level 35 dan dapat dijalankan pada perangkat Android dengan API level 30 ke atas (Android 11+). Bagian **defaultConfig** juga mendefinisikan **applicationId**, **versionCode**, dan **versionName**, yang diperlukan dalam distribusi dan pembaruan aplikasi di Google Play. **testInstrumentationRunner** disetel ke **androidx.test.runner.AndroidJUnitRunner** untuk keperluan pengujian instrumentasi.

Dalam blok **buildTypes**, terdapat konfigurasi **release** yang menonaktifkan proses **minifyEnabled**, artinya kode tidak akan dimampatkan saat proses rilis, dan **proguardFiles** menunjukkan penggunaan konfigurasi default ProGuard serta file **proguard-rules.pro** untuk aturan kustom. Kemudian, **compileOptions** dan **kotlinOptions** menetapkan bahwa proyek ini menggunakan Java 11 baik sebagai target maupun sumber kompatibilitas, serta mengarahkan Kotlin untuk menargetkan JVM versi 11.

Selanjutnya, fitur **viewBinding** diaktifkan melalui **buildFeatures**, yang memungkinkan pengaksesan langsung ke elemen UI tanpa menggunakan **findViewById**, sehingga meningkatkan keamanan dan efisiensi pengembangan.

Pada bagian **dependencies**, berbagai pustaka penting ditambahkan untuk menunjang pengembangan. Beberapa dependensi dideklarasikan menggunakan **Version Catalog**, seperti **libs.androidx.core.ktx**, **libs.androidx.appcompat**, **libs.material**, dan **libs.androidx.activity**, yang mengacu pada konfigurasi versi di file **libs.versions.toml**. Sementara itu, pustaka **RecyclerView**, **ConstraintLayout**, serta pustaka untuk **lifecycle-viewmodel-ktx**, **lifecycle-livedata-ktx**, dan **fragment-ktx**

ditambahkan secara eksplisit. Untuk mendukung pengujian, digunakan **junit** untuk unit testing dan **androidx.junit** serta **espresso.core** untuk instrumented testing.

Debugging merupakan proses identifikasi dan perbaikan kesalahan (bug) dalam sebuah program. Pada Android Studio, pengguna dapat menetapkan breakpoint pada baris kode tertentu untuk menghentikan sementara eksekusi program, sehingga memungkinkan pemeriksaan nilai variabel dan alur eksekusi secara detail. Terdapat tiga fitur utama dalam proses debugging yang sangat penting, yaitu *Step Over*, *Step Into*, dan *Step Out*. *Step Over* berfungsi untuk melewati pemanggilan fungsi tanpa memasuki isi fungsi tersebut, yang berguna ketika pengguna hanya ingin mengetahui bahwa fungsi telah dipanggil tanpa perlu memeriksa implementasinya secara rinci. Sebagai contoh, saat menekan *Step Over* pada baris pemanggilan fungsi, debugger akan langsung melanjutkan ke baris berikutnya tanpa memasuki fungsi yang dipanggil. Sebaliknya, *Step Into* memungkinkan pengguna untuk masuk ke dalam fungsi yang sedang dipanggil sehingga setiap baris kode di dalam fungsi tersebut dapat ditelusuri secara detail. Apabila pengguna sudah berada dalam sebuah fungsi dan ingin kembali ke fungsi pemanggil, fitur *Step Out* digunakan untuk keluar dari fungsi saat ini dan melanjutkan eksekusi di luar fungsi tersebut. Shortcut keyboard yang umum digunakan untuk ketiga fitur ini adalah **F8** untuk *Step Over*, **F7** untuk *Step Into*, dan **Shift + F8** untuk *Step Out* pada sistem operasi Windows maupun Mac.

D. Tautan Git

<https://github.com/biilaaa/Pemrograman-Mobile>

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Dalam arsitektur aplikasi Android, kelas **Application** adalah komponen penting yang berfungsi sebagai titik masuk global untuk status dan konfigurasi aplikasi. Kelas ini merupakan subclass dari **android.app.Application** dan hanya ada satu instance **Application** yang dibuat oleh sistem saat proses aplikasi dimulai, sebelum aktivitas, layanan, atau komponen lain diinisialisasi. Fungsi Utama Application Class:

1. Inisialisasi Global

Kelas **Application** ideal digunakan untuk melakukan setup global yang dibutuhkan oleh seluruh aplikasi, seperti:

- Inisialisasi dependency injection (misalnya Hilt, Dagger, Koin)
- Inisialisasi pustaka pihak ketiga (misalnya Firebase, Timber, Retrofit)
- Menentukan konfigurasi logging atau analitik
- Membuat instance singleton atau cache global

2. Menyediakan Konteks Aplikasi

Kelas ini menyediakan **applicationContext**, yaitu konteks global yang tidak terkait dengan lifecycle aktivitas atau fragment mana pun. Ini berguna untuk komponen yang butuh konteks, tetapi tidak terikat pada UI.

3. Monitoring Lifecycle Aplikasi

Sejak API 14 (Android 4.0), **Application** dapat memonitor lifecycle aktivitas menggunakan **Application.ActivityLifecycleCallbacks**. Ini memungkinkan pengembang melacak kapan aktivitas dimulai, dihentikan, atau dihancurkan — berguna untuk analitik, pelacakan penggunaan, atau manajemen status global.

4. Pengelolaan Resource Global

Application digunakan untuk menyimpan data global (misalnya token autentikasi, pengaturan pengguna) yang bisa diakses dari mana saja selama proses aplikasi berjalan. Namun, perlu hati-hati agar tidak menyebabkan memory leak atau pelanggaran prinsip modularitas.